



Unidad 1: Fundamentos de algoritmos y estructuras de datos

Nombre: Carlos Antonio Alvarez Angulo

Matrícula: 00366182

Grupo: Grupo 451

Fecha: 09/09/2025

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5
6  // -----
7  // Definición de estructuras
8  // -----
9  typedef struct _tdato {
10     int dato;
11     struct _tdato *sig;
12 } sTdato;
13
14 typedef sTdato *Tdato;
15
16 typedef struct {
17     char nombre[30];
18     int edad;
19     char matricula[15];
20 } Alumno;
21
22 // -----
23 // Prototipos
24 // -----
25 void menuPrincipal();
26 void ejercicio1();
27 void ejercicio2();
28 void ejercicio3();
29 void ejercicio4();
30
31 // Funciones para pila dinámica (Ej 2 y Ej 3)
32 int push(Tdato *pila, int valor);
33 int pop(Tdato *pila);
34
35 // Funciones para pila con Alumno (Ej 3)
36 int pushAlumno(Tdato *pila, Alumno a);
37 Alumno popAlumno(Tdato *pila);
38 void mostrarAlumno(Alumno a);
39
40 // Validación de entrada
41 int leerEnteroEnRango(const char *mensaje, int min, int max);
42
43 // -----
44 // MAIN
45 // -----
46 int main() {
47     menuPrincipal();
48     return 0;
49 }
```



Universidad Autónoma de Baja California

Algoritmos y Estructura de Datos

```
49 }
50
51 // -----
52 // Menú Principal
53 // -----
54 void menuPrincipal() {
55     int opc;
56     do {
57         system("CLS");
58         printf("\n===== MENU PRINCIPAL =====\n");
59         printf("1. Pila Estatica (PUSH/POP)\n");
60         printf("2. Pila Dinamica (Enteros)\n");
61         printf("3. Pila con datos de Alumno\n");
62         printf("4. Factorial con Pilas\n");
63         printf("5. Salir\n");
64         opc = leerEnteroEnRango("Seleccione una opcion: ", 1, 5);
65
66         switch(opc) {
67             case 1: ejercicio1(); break;
68             case 2: ejercicio2(); break;
69             case 3: ejercicio3(); break;
70             case 4: ejercicio4(); break;
71             case 5: printf("\nSaliendo...\n"); break;
72         }
73     } while(opc != 5);
74     system("pause");
75 }
76
77 // -----
78 // EJERCICIO 1: Pila Estatica
79 // -----
80 void ejercicio1() {
81     int pila[100], tope = -1;
82     int opc, valor;
83
84     do {
85         system("CLS");
86         printf("\n--- EJERCICIO 1: PILA ESTATICA ---\n");
87         printf("1. PUSH\n");
88         printf("2. POP\n");
89         printf("3. SALIR\n");
90         opc = leerEnteroEnRango("Opcion: ", 1, 3);
91
92         switch(opc) {
```



Universidad Autónoma de Baja California

Algoritmos y Estructura de Datos

```
93     case 1:
94         if (tope < 99) {
95             printf("Ingrese valor: ");
96             scanf("%d", &valor);
97             pila[++tope] = valor;
98         } else {
99             printf("Pila llena!\n");
100         }
101         break;
102     case 2:
103         if (tope >= 0) {
104             printf("POP: %d\n", pila[tope--]);
105         } else {
106             printf("Pila vacia!\n");
107         }
108         break;
109     case 3: printf("Regresando al menu principal...\n"); break;
110 }
111 system("pause");
112 } while(opc != 3);
113 }
114
115 // -----
116 // EJERCICIO 2: Pila Dinamica Enteros
117 // -----
118 void ejercicio2() {
119     Tdato pila = NULL;
120     int opc, valor;
121
122     do {
123         system("CLS");
124         printf("\n--- EJERCICIO 2: PILA DINAMICA ---\n");
125         printf("1. PUSH\n");
126         printf("2. POP\n");
127         printf("3. SALIR\n");
128         opc = leerEnteroEnRango("Opcion: ", 1, 3);
129
130         switch(opc) {
131             case 1:
132                 printf("Ingrese valor: ");
133                 scanf("%d", &valor);
134                 if (!push(&pila, valor)) {
135                     printf("Error: no se pudo asignar memoria.\n");
```



Universidad Autónoma de Baja California
Algoritmos y Estructura de Datos

```
135         printf("Error: no se pudo asignar memoria.\n");
136     }
137     break;
138     case 2:
139         if (pila) {
140             printf("POP: %d\n", pop(&pila));
141         } else {
142             printf("Pila vacia!\n");
143         }
144         break;
145     case 3:
146         while(pila) pop(&pila);
147         printf("Memoria liberada. Regresando...\n");
148         break;
149     }
150     system("pause");
151 } while(opc != 3);
152 }
153
154 // -----
155 // EJERCICIO 3: Pila con datos de Alumno
156 // -----
157 void ejercicio3() {
158     Tdato pila = NULL;
159     int opc;
160     Alumno a;
161
162     do {
163         system("CLS");
164         printf("\n--- EJERCICIO 3: PILA ALUMNOS ---\n");
165         printf("1. PUSH (Agregar alumno)\n");
166         printf("2. POP (Eliminar alumno)\n");
167         printf("3. SALIR\n");
168         opc = leerEnteroEnRango("Opcion: ", 1, 3);
169
170         switch(opc) {
171             case 1:
172                 printf("Nombre: ");
173                 getchar(); // limpiar buffer
174                 fgets(a.nombre, sizeof(a.nombre), stdin);
175                 a.nombre[strcspn(a.nombre, "\n")] = 0;
176
177                 a.edad = leerEnteroEnRango("Edad: ", 1, 120);
```



```
178
179     printf("Matricula: ");
180     scanf("%14s", a.matricula);
181
182     if (!pushAlumno(&pila, a)) {
183         printf("Error: no se pudo asignar memoria.\n");
184     }
185     break;
186
187     case 2:
188         if (pila) {
189             mostrarAlumno(popAlumno(&pila));
190         } else {
191             printf("Pila vacia!\n");
192         }
193         break;
194
195     case 3:
196         while(pila) popAlumno(&pila);
197         printf("Memoria liberada. Regresando...\n");
198         break;
199     }
200     system("pause");
201 } while(opc != 3);
202
203 // -----
204 // EJERCICIO 4: Factorial usando Pilas
205 // -----
206 void ejercicio4() {
207     int n;
208     char continuar;
209     do {
210         system("CLS");
211         n = leerEnteroEnRango("\nIngrese un numero para calcular su factorial (>=1): ", 1, 100);
212
213         Tdato pila = NULL;
214         for (int i = n; i > 0; i--) {
215             if (!push(&pila, i)) {
216                 printf("Error: no se pudo asignar memoria.\n");
217                 return;
218             }
219         }
220     } while(continuar == 's');
```



```
218     }
219
220     int resultado = 1;
221     printf("\nProceso:\n");
222     while(pila) {
223         int val = pop(&pila);
224         resultado *= val;
225         printf("%d -> resultado parcial: %d\n", val, resultado);
226     }
227
228     printf("Factorial de %d = %d\n", n, resultado);
229
230     printf("\nDesea calcular otro factorial? (s/n): ");
231     getchar();
232     continuar = getchar();
233     system("pause");
234     } while(continuar == 's' || continuar == 'S');
235 }
236
237 // -----
238 // Funciones auxiliares
239 // -----
240 int push(Tdato *pila, int valor) {
241     Tdato nuevo = (Tdato)malloc(sizeof(sTdato));
242     if (!nuevo) return 0; // fallo malloc
243     nuevo->dato = valor;
244     nuevo->sig = *pila;
245     *pila = nuevo;
246     return 1;
247 }
248
249 int pop(Tdato *pila) {
250     if (*pila == NULL) return -1;
251     Tdato temp = *pila;
252     int valor = temp->dato;
253     *pila = temp->sig;
254     free(temp);
255     return valor;
256 }
257
258 int pushAlumno(Tdato *pila, Alumno a) {
259     Tdato nuevo = (Tdato)malloc(sizeof(sTdato) + sizeof(Alumno));
260     if (!nuevo) return 0;
261     nuevo->sig = *pila;
```



```
262     memcpy(&(nuevo->dato), &a, sizeof(Alumno));
263     *pila = nuevo;
264     return 1;
265 }
266
267 Alumno popAlumno(Tdato *pila) {
268     Alumno a;
269     Tdato temp = *pila;
270     memcpy(&a, &(temp->dato), sizeof(Alumno));
271     *pila = temp->sig;
272     free(temp);
273     return a;
274 }
275
276 void mostrarAlumno(Alumno a) {
277     printf("Alumno: %s | Edad: %d | Matricula: %s\n", a.nombre, a.edad, a.matricula);
278 }
279
280 int leerEnteroEnRango(const char *mensaje, int min, int max) {
281     int valor;
282     char c;
283     while (1) {
284         printf("%s", mensaje);
285         if (scanf("%d", &valor) == 1 && valor >= min && valor <= max) {
286             return valor;
287         } else {
288             printf("Entrada invalida. Intente de nuevo.\n");
289             while ((c = getchar()) != '\n' && c != EOF);
290         }
291     }
292 }
293
```

--- EJERCICIO 1: PILA ESTATICA ---

1. PUSH

2. POP

3. SALIR

Opcion: 1

Ingrese valor: 7

Presione una tecla para continuar . . . █

===== MENU PRINCIPAL =====

1. Pila Estatica (PUSH/POP)

2. Pila Dinamica (Enteros)

3. Pila con datos de Alumno

4. Factorial con Pilas

5. Salir

Seleccione una opcion: █